深く積む

残差と LayerNorm — 勾配を守って、GPT の全身を組む

rkskmt

前回の振り返り

- **マルチヘッド注意**（会議で集める）と **FFN**（席に戻って考える）ができた
- どちらも **入口と出口が同じ形** — 理屈の上では、いくらでも積み重ねられる

> **クイズ**
>
> 同じ形だから8段でも96段でも積める。では素朴に積めば、積んだぶんだけ賢くなる？

- ならない。深さには税金が2つ：**勾配が入口まで届かない** ／ **途中のベクトルの大きさが暴れる**
- 今日はこの2つに保険をかけてから、GPT の全身を組む

前回の部品を持ってくる

- 今日のコードは前回の **`Head`・`MultiHead`・`FeedForward`** の続き。自分のファイルの続きに書くか、下をコピーして先頭に置く

▸ 前回の部品（Head / MultiHead / FeedForward）

バックプロパゲーションを30秒で

- 学習に必要なのは「各パラメータを動かすと損失がどう動くか」＝**勾配**。**`loss.backward()`** が自動で出してくれるやつ
- 深いネットは変換の連なり。**勾配は、通り道にある傾きの掛け算**で入口まで運ばれる（連鎖律 **chain rule**）

$$\text{入口に届く勾配} = (\text{部品C}\text{の傾き}) \times (\text{部品B}\text{の傾き}) \times (\text{部品A}\text{の傾き})$$

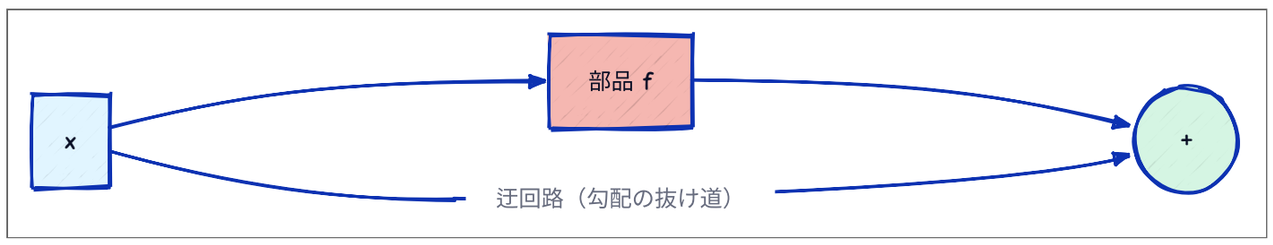
- 例：どの部品も傾きが 0.5 なら、3段で $0.5^3 = 0.125$、**10段で** $0.5^{10} \approx 0.001$
- **1より小さい傾きが続くだけで、入口の勾配は千分の1に痩せる** — **勾配消失**。深いネットの持病

保険1：迂回路を付ける — 残差接続

- 対策：部品を**迂回路つき**にする

$$x \leftarrow x + f(x)$$

- 出力＝「元の x」＋「部品の仕事ぶん」。勾配は足し算の枝を**素通り**して入口まで届く
- **残差接続**（residual connection）。コードでは **`x + f(x)`** と書くだけ



- 「積む」が「元の情報に**修正を足していく**」に変わる — 深くしても壊れにくい設計

保険2：入力を揃えてから渡す — LayerNorm

- 深く積むと、途中のベクトルの大きさがバラバラに育って学習が不安定になる
- 各位置のベクトルを**平均0・ばらつき1**に揃える部品を、各部品の直前に置く

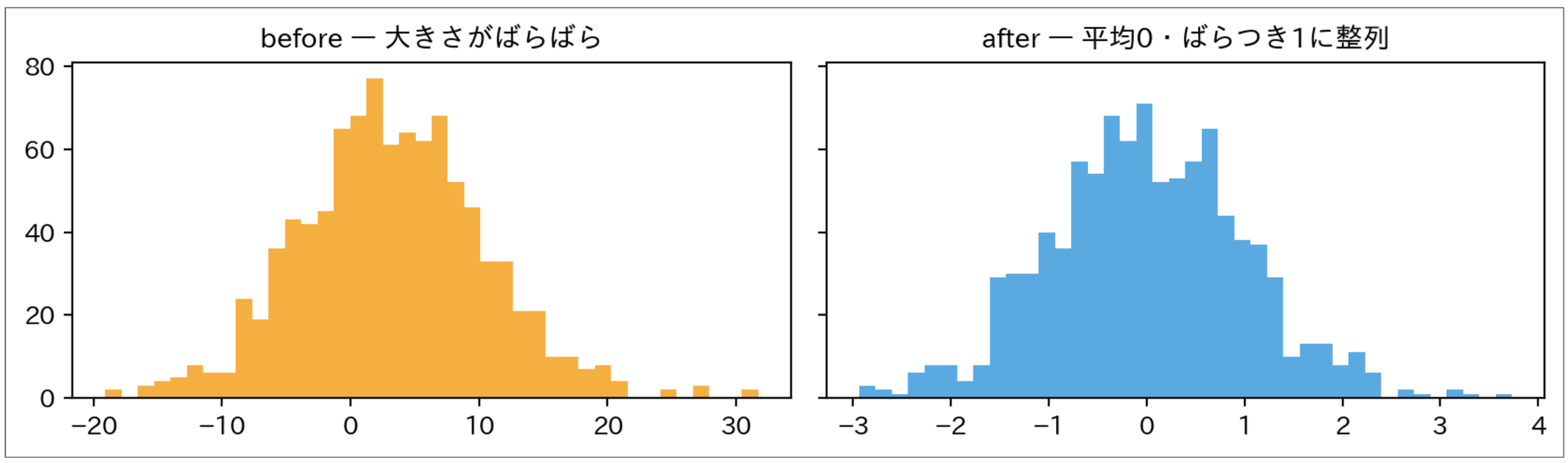
Code

```python
import torch
import torch.nn as nn

torch.manual_seed(0)
x = torch.randn(2, 5, 96) * 7 + 3          # 平均3・ばらつき7のベクトルたち
ln = nn.LayerNorm(96)
y = ln(x)
print(f"before: mean {x.mean():.2f} / std {x.std():.2f}")
print(f"after : mean {y.mean():.2f} / std {y.std():.2f}")
```

Result

```
before: mean 3.27 / std 7.24
after : mean -0.00 / std 1.00
```



- **LayerNorm**（layer normalization）— 「測る前に平均0・ばらつき1へ揃える」**標準化**の仲間（テストの素点を偏差値に直すのと同じ発想）
- 揃えたあとに掛ける・足す用の小さなパラメータ（γ・β）だけ学習される

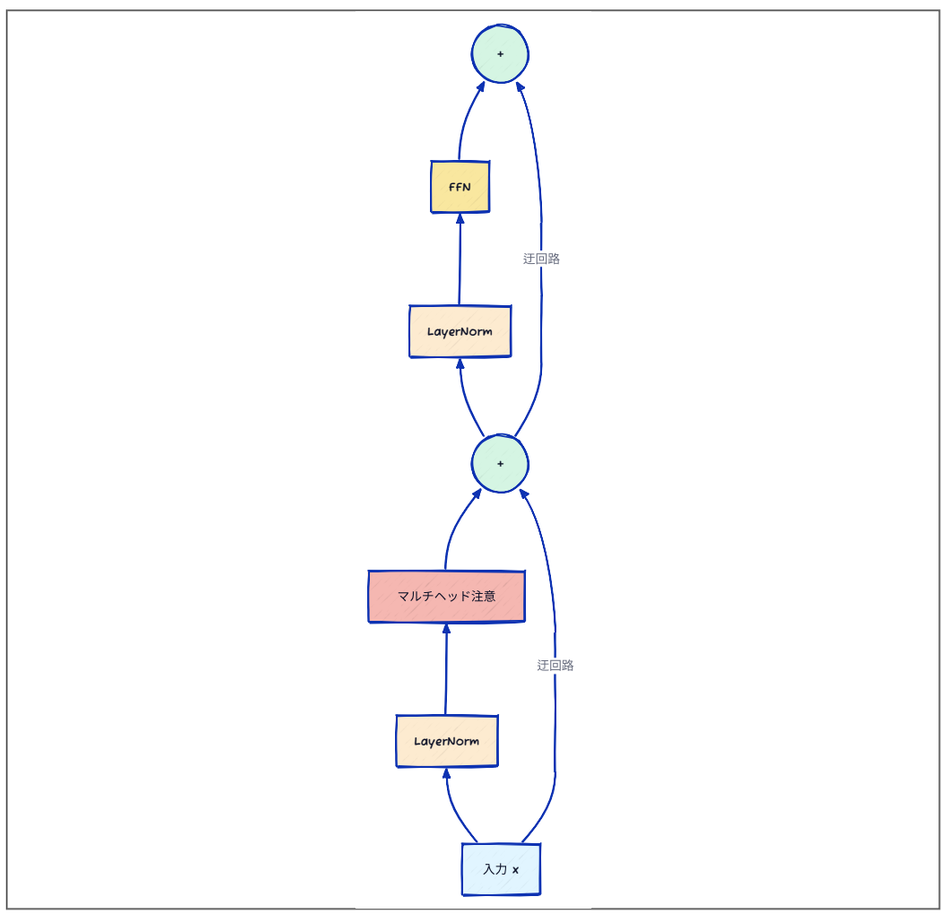
ブロック — Transformer の1段

- 4部品を決まった順に組む。この1段を**ブロック**（Transformer block）と呼ぶ

Code

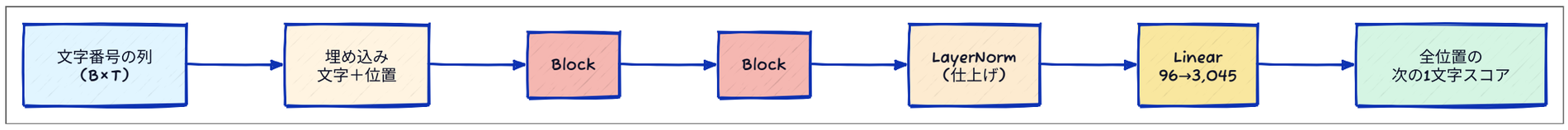
```
class Block(nn.Module):
    def __init__(self, n_embd, n_head, block_size):
        super().__init__()
        self.sa = MultiHead(n_embd, n_head, block_size)
        self.ff = FeedForward(n_embd)
        self.ln1 = nn.LayerNorm(n_embd)
        self.ln2 = nn.LayerNorm(n_embd)

    def forward(self, x):
        x = x + self.sa(self.ln1(x))    # 揃えて→会議→迂回路で合流
        x = x + self.ff(self.ln2(x))    # 揃えて→考える→合流
        return x
```



- **ブロックは入口と出口が同じ形**（T×n_embd）→ **いくらでも積み重ねられる**

全身を組む — これが GPT



Code

```
class GPT(nn.Module):
    def __init__(self, vocab_size, n_embd, n_head, n_layer, block_size):
        super().__init__()
        self.block_size = block_size
        self.tok_emb = nn.Embedding(vocab_size, n_embd)    # 文字の埋め込み
        self.pos_emb = nn.Embedding(block_size, n_embd)    # 位置の埋め込み
        self.blocks = nn.Sequential(
            *[Block(n_embd, n_head, block_size) for _ in range(n_layer)])
        self.ln_f = nn.LayerNorm(n_embd)
        self.head = nn.Linear(n_embd, vocab_size)          # 語彙スコアに戻す

    def forward(self, idx):                                # idx: (B, T) 文字番号
        B, T = idx.shape
        x = self.tok_emb(idx) + self.pos_emb(torch.arange(T))
        x = self.blocks(x)
        return self.head(self.ln_f(x))                     # (B, T, vocab) ロジット

model = GPT(vocab_size=3045, n_embd=96, n_head=4, n_layer=2, block_size=64)
logits = model(torch.randint(3045, (2, 10)))               # 2文・各10文字を試しに
print(logits.shape)
print(f"パラメータ数: {sum(p.numel() for p in model.parameters()):,}")
```

Result

```
torch.Size([2, 10, 3045])
パラメータ数: 817,125
```

- 入口は文字番号の列、出口は**全位置ぶんの「次の1文字」スコア** — バイグラムと同じ約束
- 語彙3,045（『吾輩は猫である』サイズ）で**約82万パラメータ**

中身の勘定

Code

```
parts = {"埋め込み（文字+位置）": [model.tok_emb, model.pos_emb],
         "ブロック×2": [model.blocks],
         "仕上げの語彙スコア": [model.head]}
for name, mods in parts.items():
    n = sum(p.numel() for m in mods for p in m.parameters())
    print(f"{name:12s}: {n:8,d}")
```

Result

```
埋め込み（文字+位置）  :  298,464
ブロック×2          :  223,104
仕上げの語彙スコア    :  295,365
```

- 埋め込みと語彙スコアだけで**7割超**。この体格で日本語3,045字と戦う
- GPT-2 も GPT-4 も、**この設計図のまま各部品を大きくして積んだだけ**（ブロック数十段・次元数千）

Tweak 設計図の数字で遊ぶ

- **GPTクラスのコード** の設定を変えて、形とパラメータ数を確認（訓練はしないので一瞬）

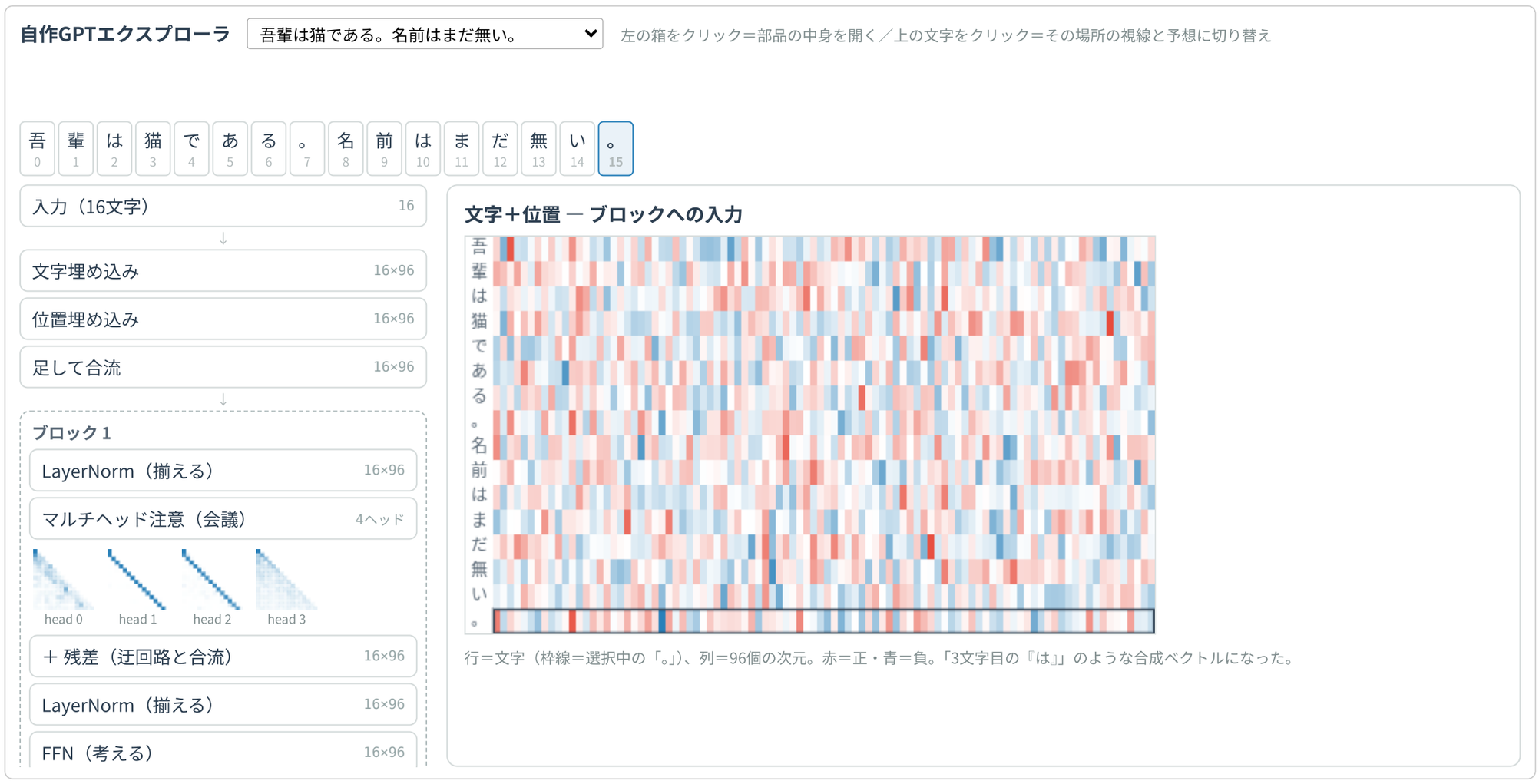
1	`n_layer=2` → `8`	パラメータはどこがどれだけ増える？（内訳の勘定で確認）
2	`n_embd=96` → `192`	層を増やすのと次元を増やすの、効き方の違いは？
3	`block_size=64` → `128`	増えるのは1部品だけ。どれ？（内訳の勘定はヒントにならない — なぜ？）

ここで観察

- `block_size` を倍にして増えるのは**位置の埋め込み表だけ**：64×96 = 6,144 → 128×96 = 12,288。パラメータ的には安い買い物（ただし各ヘッドが作る視線の表は T×T に広がるので、計算の手間は別勘定）
- 内訳の勘定で pos_emb は「埋め込み」の行に混ざっている — 増分 6,144 はそこに現れる

全身を歩く — 自作GPT

いま組んだ **GPT** クラスの中身を、訓練済みの実重みで上から下まで歩く。



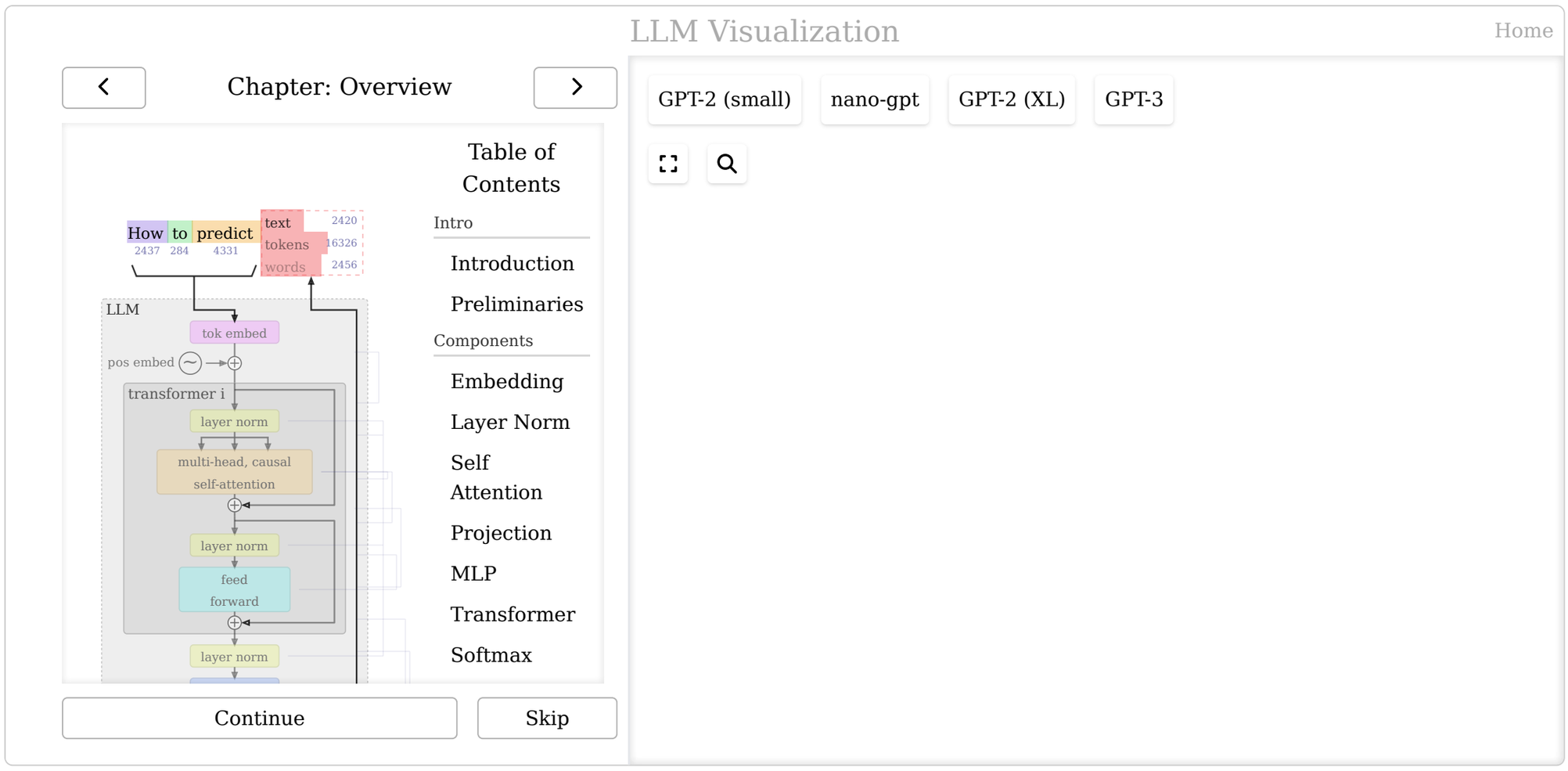
0. 左の箱を **入力から順に** クリックして、各部品の出力（ヒートマップ）を眺める
1. 「＋ 残差」の前後でヒートマップの模様が **大きくは変わらない** ことを確認 — 迂回路の実感
2. 各箱が今日の **GPT** クラスの **どの行** にあたるか、コードと突き合わせる

エクスプローラを別タブで開く

本物の中を歩く



同じ設計図の**実物大**も歩ける。nanoGPT の中身を3Dで見られる公開デモ：



- 自作GPTの箱と対応する部品を**3つ**見つけて、隣の人に30秒で説明（Embedding／Attention／MLP＝FFN…）
- ブラウザ内で GPT-2 を動かせる **Transformer Explainer** も同じ遊びができる／**LLM Visualization を別タブで開く**

> ⓘ **ノート**
>
> どちらのデモにも、今日までに自作していない部品は（ほぼ）出てこない。 「知らない箱」が出たら、それはたいてい名前が違うだけ — 対応表を作れたら勝ち。

まとめ

- 深さの税金は2つ：勾配消失（傾きの掛け算で痩せる）とベクトルの暴れ
- **残差**＝ $x + f(x)$ の迂回路で勾配を守る。**LayerNorm**＝部品の前でベクトルを揃える
- ブロック＝「揃えて会議、揃えて自習、どちらも迂回路つき」。同じ形で積み放題
- 埋め込み→ブロック×N→語彙スコアで **GPT の全身が完成**。語彙まわりがパラメータの7割

現在の設計図　文字＋位置 → [LN→注意→＋] [LN→FFN→＋] ×N → 語彙スコア

> ⓘ **次回予告**
>
> 設計図はできた。次回、『吾輩は猫である』32万字を流し込んで**火を入れる**。 デタラメしか言えない82万個の数が、言葉を覚えていく様子を定点観測する。

練習問題

以下の問題を**Pythonコード**で解け。

問題1：迂回路の効果を勾配で確かめる

残差接続の有無で入口に届く勾配がどう変わるか、ミニ実験で確かめよ。

1. `f = nn.Sequential(nn.Linear(16, 16), nn.ReLU())` を10個作り、`x = torch.randn(1, 16, requires_grad=True)` を用意
2. **残差なし**（`x` に順に `f` を適用）と**残差あり**（`x + f(x)` を繰り返す）の2通りで出力の合計を `backward()`
3. `x.grad.abs().mean()` を比較（残差ありの方が桁違いに大きいはず）

問題2：語彙のコスト

`vocab_size` を 3,045（文字レベル日本語）と 65（英語の文字レベル相当）にした GPT のパラメータ数を比べよ。差はどの部品から来るか、内訳の勘定で示せ。

解答例

▸ 解答例を見る

Result

```
残差なしの入口勾配: 0.000035
残差ありの入口勾配: 6.937421
vocab  3045: 全体  817,125（うち語彙まわり 587,685）
vocab    65: 全体  241,985（うち語彙まわり  12,545）
```

- 問題1：ReLU 10段の掛け算で勾配は目に見えて痩せる（30秒リキャップの 0.5^{10} が現物で出る）。迂回路があると入口まで太いまま届く
- 問題2：ブロック部分は同じなのに全体は3倍近く違う — **大語彙のコストは埋め込みと出口に集中**する。実物の LLM が文字でなくサブワード（BPE）を使う理由のひとつ